

Інструментальні засоби розробки програмного забезпечення

Лабораторна робота №1

Практичне застосування системи контролю версій Git та юніт-тестування у процесі розробки програмного забезпечення.

Виконала студентка 2-го курсу

Групи ІПС-21

Лук'янченко Єлізавета Юріївна

Теоретичний вступ

Система контролю версій Git дозволяє розробникам відслідковувати зміни у вихідному коді, працювати над різними гілками проекту незалежно, а також ефективно співпрацювати у команді.

Сервіс GitHub є хмарною платформою для розміщення Git-репозиторіїв, що надає можливість організовувати командну розробку, створювати Pull Request'и, проводити рев'ю коду.

Юніт-тестування — це процес перевірки правильності роботи окремих модулів або функцій програми. Зазвичай для цього застосовуються фреймворки тестування (Google Test).

Метою тестування є підвищення якості коду, виявлення помилок на ранніх етапах і забезпечення стабільності при внесенні змін.

Основна мета використання Google Test — перевірити коректність роботи функцій, класів і модулів програми без необхідності запускати всю систему.

Хід виконання роботи

Для виконання лабораторної роботи було використано проект, що реалізує структури даних ArrayList та LinkedList, а також алгоритми сортування.

Створено новий репозиторій на GitHub:

<https://github.com/LukiancenkoL/IZR-sem3lab1.git>

Створено окрему гілку для тестування: unit-tests.

До репозиторію додано окрему папку з тестами “tests”:

array_list_test.cpp, linked_list_test.cpp, insertion_sort_test.cpp, merge_sort_test.cpp, quick_sort_test.cpp.

У процесі роботи виконано кілька комітів із логічними назвами:

- add tests for ArrayList
- fix ArrayList insert() and remove()
- add test for ArrayList private method reallocate
- add tests for LinkedList
- fix LinkedList::pop_front()
- fix LinkedList::insert()
- fix LinkedList::remove()
- add tests for InsertionSort
- add tests for MergeSort
- fix MergeSort
- add tests for QuickSort

Створено Pull Request після завершення розробки тестів із гілки unit-tests до main.

Для зручності тестування у коді було додано умову перевірки параметрів командного рядка, щоб забезпечити можливість запуску юніт-тестів за допомогою передання аргументу --test.

Написання юніт-тестів

Кожен тест у Google Test описується за допомогою макросу TEST():

```
TEST(GroupName, TestName) {  
    // Тіло тесту  
}, де  
    GroupName — логічне об'єднання тестів (наприклад, array_list,  
    sort);  
    TestName — описує конкретну функцію або поведінку, що  
    перевіряється (наприклад, push_back, insertion_sort).
```

Google Test надає набір макросів для різних типів перевірок:

ASSERT_EQ(a, b) - Перевіряє, що $a == b$. У разі помилки тест завершується.

EXPECT_EQ(a, b) - Перевіряє рівність, але дозволяє продовжити тест.

ASSERT_NE(a, b) - Перевіряє, що $a != b$.

ASSERT_TRUE(cond) - Перевіряє, що умова істинна.

ASSERT_FALSE(cond) - Перевіряє, що умова хибна.

ASSERT_THROW(expr, exception_type) - Очікує, що під час виконання expr буде згенеровано виняток певного типу.

ASSERT_NO_THROW(expr) - Перевіряє, що під час виконання не виникне виняток.

FRIEND_TEST - Використовується для надання тестовим функціям доступу до приватних і захищених членів класу.

Це потрібно у випадках, коли необхідно протестувати внутрішню логіку класу, яка недоступна ззовні через інкапсуляцію.

Висновки

У ході лабораторної роботи було:

набуто практичні навички роботи з Git і GitHub;

опановано створення гілок, комітів, та Pull Request;

реалізовано повний цикл командної розробки: створення, тестування, рецензія коду;

написано набір юніт-тестів із використанням Google Test;

закріплено знання про структури даних та алгоритми сортування.